Algorithms, 2026 Fall: Homework 2

Yifan Chen

Problem 3

We say that a vertex v in an undirected graph G is an *articulation point* if its removal increases the number of connected components of G.

(1) Try to detect whether v is an articulation point using DFS.

(2) Try to find all articulation points using DFS (your algorithm should run in $O(|V| + |E|)$ time).

Answer:

We directly solve the part (2) using the Tarjan algorithm (which also answers part (1)). Let $\mathrm{dfn}(x)$ denote the time stamp of visiting x in the DFS, and let

$$\mathrm{low}(x) = \min\Big(\{\mathrm{dfn}(x)\} \cup \{\mathrm{dfn}(t) : t \text{ is adjacent to some node } y \text{ in the subtree of } x,\ t \neq \mathrm{fa}(y)\}\Big).$$

These variables can be determined in $O(|V|+|E|)$ time by a single DFS from any vertex r (if G is disconnected, restart from every unvisited vertex), then a vertex v is an articulation point if and only if:

- $v = r$ and v has at least two children in the DFS tree;
- $v \neq r$ and v has a child x in the DFS tree with $\mathrm{low}(x) \geq \mathrm{dfn}(v)$.

Problem 4

(Source: KT book pp. 110, Problem 9.)

Suppose that an n-node undirected graph $G = (V, E)$ contains two nodes s and t such that the distance between s and t is strictly greater than $n/2$. Show that there must exist some node v, not equal to either s or t, such that deleting v from G destroys all s-t paths. (In other words, the graph obtained from G by deleting v contains no path from s to t.) Give an algorithm with running time $O(m+n)$ to find such a node v.

Answer:

We build a shortest path tree from s, and mark the chain from s to t as the main chain. We do not need to consider the vertices not in the chain since deleting any of them will not affect the main chain.

For any other path from s to t, it must contain some vertices not in the main chain. Assume w.l.o.g. that the path is $s \to u \to v \to t$, where $s \to u$ and $v \to t$ are on the main chain, while $u \to v$ does not use the vertices in the main chain. Then we know that the vertices between u and v (not included) cannot be chosen. However, since the main chain is a shortest path, once we exclude any vertex, there must be at least one new vertex out of the main chain. Since the main chain has more than $n/2 + 1$ vertices, there must be some vertex other than s and t that is not excluded, and any of them satisfies the requirement.

The algorithm runs in $O(m + n)$. Use BFS to construct the shortest path DAG, i.e. the edges (u, v) with $\text{dist}(s, v) = \text{dist}(s, u) + 1$, and choose one parent in this DAG for each node, so that the tree path from s to t is the main chain. Process the nodes in increasing distance and compute for every node x the last main-chain vertex l_x that can reach x, together with the first main-chain vertex r_x reachable from x. For every x outside the chain, mark the open interval (l_x, r_x) on the chain by a difference array; taking prefix sums then marks exactly the excluded vertices, and any vertex of the chain other than s and t that remains unmarked is the required v.

Problem 5

(Source: KT book pp. 193, Problem 12.)

Suppose you have n video streams that need to be sent, one after another, over a communication link. Stream i consists of a total of b_i bits that need to be sent, at a constant rate, over a period of t_i seconds. You cannot send two streams at the same time, so you need to determine a schedule for the streams: an order in which to send them. Whichever order you choose, there cannot be any delays between the end of one stream and the start of the next. Suppose your schedule starts at time 0 (and therefore ends at time $\sum_{i=1}^{n} t_i$, whichever order you choose). We assume that all the values b_i and t_i are positive integers.

Now, because you're just one user, the link does not want you taking up too much bandwidth, so it imposes the following constraint, using a fixed parameter r:

> $(*)$ For each natural number $t > 0$, the total number of bits you send over the time interval from 0 to t cannot exceed rt.

Note that this constraint is only imposed for time intervals that start at 0, not for time intervals that start at any other value. We say that a schedule is *valid* if it satisfies the constraint $(*)$.

Example. Suppose we have $n = 3$ streams, with $(b_1, t_1) = (2000, 1)$, $(b_2, t_2) = (6000, 2)$, $(b_3, t_3) = (2000, 1)$, and suppose the link's parameter is $r = 5000$. Then the schedule that runs the streams in the order $1, 2, 3$ is valid, since the constraint $(*)$ is satisfied:

- $t = 1$: the whole first stream has been sent, and $2000 < 5000 \cdot 1$;
- $t = 2$: half of the second stream has also been sent, and $2000 + 3000 < 5000 \cdot 2$.

Similar calculations hold for $t = 3$ and $t = 4$.

(a) Consider the following claim.

> **Claim:** There exists a valid schedule if and only if each stream i satisfies $b_i \leq rt_i$.

Decide whether you think the claim is true or false, and give a proof of either the claim or its negation.

(b) Give an algorithm that takes a set of n streams, each specified by its number of bits b_i and its time duration t_i, as well as the link parameter r, and determines whether there exists a valid schedule. The running time of your algorithm should be polynomial in n.

Answer:

For part (a), the claim is clearly wrong: let $n = 2, r = 10$ with $(b_1, t_1) = (5, 1)$ and $(b_2, t_2) = (15, 1)$. In this case, $1, 2$ is a valid schedule while $b_2 > rt_2$.

For part (b), there exists a valid schedule if and only if $\sum_i b_i \leq r \sum_i t_i$. We sort the streams by b_i/t_i in increasing order, thus the streams with $b_i \leq rt_i$ come first, so the prefix sums of $b_i - rt_i$ stay non-positive before increasing to $\sum_i (b_i - rt_i)$. Therefore, only the global sum should be checked.

Problem 7

(Source: KT book pp. 198, Problem 19.)

A group of network designers at the communications company CluNet find themselves facing the following problem. They have a connected graph $G = (V, E)$, in which the nodes represent sites that want to communicate. Each edge e is a communication link, with a given available bandwidth b_e.

For each pair of nodes $u, v \in V$, they want to select a single u-v path P on which this pair will communicate. The *bottleneck rate* $b(P)$ of this path P is the minimum bandwidth of any edge it contains; that is, $b(P) = \min_{e \in P} b_e$. The best achievable bottleneck rate for the pair u, v in G is simply the maximum, over all u-v paths P in G, of the value $b(P)$.

It's getting to be very complicated to keep track of a path for each pair of nodes, and so one of the network designers makes a bold suggestion: maybe one can find a spanning tree T of G so that for every pair of nodes u, v, the unique u-v path in the tree actually attains the best achievable bottleneck rate for u, v in G. (In other words, even if you could choose any u-v path in the whole graph, you couldn't do better than the u-v path in T.)

Show that such a tree exists, and give an efficient algorithm to find one. That is, give an algorithm constructing a spanning tree T in which, for each $u, v \in V$, the bottleneck rate of the u-v path in T is equal to the best achievable bottleneck rate for the pair u, v in G.

Answer:

We can sort the edges by b_e in decreasing order, and try adding them into an initially empty tree T. In other words, we find the maximum spanning tree of G. This can be done by Kruskal's algorithm in $O(m \log m)$.

Now consider any pair u, v, we find the minimum value w on their path on the tree. By the process of Kruskal, we know that u, v are not connected using only edges with $b_e > w$, which proves the tree T is optimal.

AI Disclosure

I used an AI assistant on all problems in this homework, for grammar polishing, LaTeX typesetting, and checking the correctness of my arguments. All mathematical ideas and proofs are my own.